

KitchenBuddy

Presentation script, with a live demo

Group project · September 2026

No slides. One laptop on the projector, five voices. Audience: the Head of Innovation.

How to use this script

Eight beats, five voices: about five minutes of speaking and a minute and a half of waiting for the laptop, six and a half minutes end to end. The waits are deliberate, because they are where the explanations go. Part 2 is the script; everything after it is preparation and backup, not more speech.

Rehearse with a stopwatch and do not add anything. If the slot is a hard five minutes, the receipt (beat 3) is the one to drop: describe it in one sentence instead and go straight from the model to "Bob, I'm drained". That version lands at five minutes exactly. If you are running long at beat 6, skip the spoken question.

Who speaks. Scrum Master opens and closes. AI Researcher takes the model and the cost of training. Data Engineer takes the data, spoken over the wait while the recipe is written. ML Engineer takes the checks. DevOps drives the laptop throughout and speaks the voice commands.

How to say it. Learn the shape of each beat and use your own words. Assume nobody in the room builds these systems: every technical word is explained where it appears, or avoided. Part 3 gives a one-line explanation of each, for questions.

Two rules. Do not narrate what you are about to do, do it and explain while it runs. Never apologise for the laptop being slow: that pause is scripted.

#	Beat	Speaker	Time	Live on screen
1	The evening this is for	Scrum Master	0:35	Today
2	The machine, and how we chose the model	AI Researcher	0:55	spoken
3	A receipt becomes the kitchen	Data Engineer	1:20	camera, then the read receipt
4	Ask it for dinner: where the recipe comes from	Data Engineer	1:05	the pot filling, then the recipe
5	Why you can trust what came back	ML Engineer	0:40	the recipe
6	Cooking, hands-free	DevOps Engineer	0:45	cooking mode
7	Why we did not train it	AI Researcher	0:35	cooking mode stays open
8	Where it stands	Scrum Master	0:25	Today

Run of show. Speech is timed at a normal pace; beats 3, 4 and 6 include the laptop's own working time, about half a minute each.

Stagecraft

- **Stand on the laptop's side** so nobody crosses the screen. The driver stays put; the others step in for their beat and back out after it.
- **Hand over while the next person is already moving**, by name: "so Léa will show you where the recipes come from".
- **During a wait, look at the room, not the screen.** Nothing kills a demo faster than five people watching a progress bar.
- **A question mid-beat gets one sentence** and "I'll come back to that at the end". Do not let it reorder the demo.

- **Have the receipt in your hand before beat 3 starts**, flat and the right way up, so the camera moment is one motion.

Part 1. Before we go on

The day before

- **Do two full rehearsals on the demo Mac, with the same stock we will use on the day.** The model does not answer exactly the same way twice, so learn what it usually says.
- **Fill the kitchen for a good quick dinner.** For example rice, eggs, tomatoes, an onion, a courgette, chicken, cheese, olive oil. Avoid a near-empty kitchen: the recipe will be weak.
- **Get a real supermarket receipt**, flat, not folded, printed clearly. Also keep a sharp JPEG photo of it in Pictures, in the Receipts folder, as a backup.
- **Decide on answer prefetching** during rehearsal. If the chef is slow to answer in cooking mode, set `PREFETCH_ENABLED=false` in the backend configuration.

Thirty minutes before

1. Plug in the charger. Close every app we don't need, because the model and the voices share the 16 GB.
2. Open LM Studio. Load `gemma-4-e2b-it-mlx`, start the server, and check that the context length is 8192 and thinking is off.
3. Start the backend without auto-reload, so nothing restarts mid-demo: in `happy-bite/backend`, run `./run.sh --no-reload`. Wait for the log line "voice: warm in ...s".
4. Open the app in Chrome at `http://localhost:5173` (or `http://127.0.0.1:8000` if the frontend is built). Use "localhost", not the network IP address: the camera only works on localhost or https.
5. Connect the laptop's sound to the room speakers so the jury hears the chef. Test the microphone by saying "Bob" once.
6. Check the Wi-Fi if you plan to show the hands-free button outside cooking mode: that one uses Chrome's online recogniser. Cooking mode itself needs no internet, and the Talk button works either way.
7. Turn off Center Stage from the Video Effects menu in the macOS menu bar, otherwise the camera zooms in on faces.
8. Set Chrome to full screen and zoom to 125 % so the back row can read.
9. Go to the Today screen and leave it there.

Things we don't do on stage

- **Don't chain follow-up questions in cooking mode.** After a few exchanges the chef can drift to the wrong step (we measured it). Ask one question about the current step, then say "Bob, next".
- **Don't use "Any ingredients".** It writes dishes that need shopping, and takes longer.
- **Don't hold the receipt too close** to the camera. It has fixed focus and the photo goes blurry. Check the review screen before pressing "Use this photo".
- **Don't import an iPhone HEIC photo.** Only JPEG, PNG and WebP are read.

Part 2. The script

1. The evening this is for

Do

Stand beside the laptop, app on Today. Don't touch it yet.

Say

Seven in the evening. Half a courgette, some goat's cheese, rice, two chicken thighs to use tomorrow. You are tired, you have twenty minutes, and you are not going shopping. A recipe site cannot help: it has no idea what is on your shelves. So something gets thrown away.

KitchenBuddy knows what is in the kitchen and what is going off, writes a recipe from exactly that, and talks you through cooking it. All on this laptop, nothing leaving the room. That is the hard part.

2. The machine, and how we chose the model

Say

A MacBook Air: one chip, sixteen gigabytes of memory shared by the processor, the graphics cores and everything else. Three models must live in it at once: one writes, one listens, one speaks.

We did not start with this model. Day one used a cloud service; from day two everything ran here. First Qwen 2.5, seven billion parameters: loaded, it took most of the memory and left nothing for the voice. Then Qwen 3.5, which thinks before it answers, and once thought for twenty-six minutes. We settled on Google's Gemma 4 E2B, the LM Studio community's four-bit build for Apple's MLX. At full precision it would need around ten gigabytes of memory; compressed to four bits it runs in a few, with room for the other two.

If asked, or if there is time

- The compression is called quantisation: each number kept in four bits instead of sixteen, sixteen possible values instead of sixty-five thousand, with a small loss of precision.
- Five billion numbers in total, about half of them lookup tables, so roughly two billion do the work for each word. That is what E2B means: effective two billion.

3. A receipt becomes the kitchen

Do

Tap +, then "Scan a receipt". Hold the receipt flat, filling the dashed frame, press the shutter, check the photo, "Use this photo".

While it reads, keep talking. When the lines appear, point at one match and one marked "Check", then tap "Put ... items away".

Say

Filling the kitchen by hand is why nobody keeps apps like this, so we read the receipt. Three steps, and only the last one is AI.

First Tesseract, an open-source text reader, turns the photo into lines of text. Then each line, like "CRF LT DEMI ECR 1 L", is looked up in a database we built from Open Food Facts: four

hundred and eighty-three thousand products sold in France, three hundred and thirty thousand of them already matched to the ingredients the app knows, searched by pieces of words so "ECR" finds "écrémé". Only then does Gemma read the line together with what the lookup found, and decide. When the two disagree, the app marks the line "Check" rather than guessing for you.

If it goes wrong

Camera or photo fails: "let's use the one we took this morning", Import a receipt from Pictures, then Receipts. Reading fails entirely: "Use a sample receipt" and give the same explanation.

4. Ask it for dinner: where the recipe comes from

Do

DevOps, clearly: "Bob, I'm drained. Give me fifteen minutes." The pot fills while it works, 20 to 30 seconds.

Say

While it writes: the model needs two things it does not know. What is in your kitchen, kept in the browser and sent with every request. And real recipes to lean on, from a public dataset, CookingRecipes by Kaiser1308 on Hugging Face: two point one million recipes, each with its source.

We indexed those once, like the index at the back of a book, in SQLite. For every request we search on the ingredients in the kitchen, keep the best few, and put them into the prompt beside the rules and the stock. The model writes from that. Nothing was trained in; it was looked up.

If asked, or if there is time

- The name for this is retrieval-augmented generation. Our search is ordinary word search with BM25 ranking, not embeddings: embedding two million recipes would take hours on a laptop, and word search answers in milliseconds.

5. Why you can trust what came back

Do

The recipe is open. Scroll once, slowly: ingredients, then steps.

Say

A model this small makes specific mistakes, so nothing it writes reaches the screen unchecked. It answers into a fixed form, boxes rather than an essay, and ordinary code checks it: do these ingredients exist in your kitchen, could a person eat this quantity, do the times add up to fifteen minutes, is the rice ever actually cooked. We added that last one after it toasted raw rice and served it.

If a check fails it gets one more go with the exact list. If it still fails, the problem is printed on the recipe rather than hidden.

6. Cooking, hands-free

Do

Press "Start cooking", let the chef read step one aloud, then say "Bob, next".
Tap the Talk button and ask, without the name: "how do I know it's ready?"

Say

Hands covered in flour, so everything is voice. For listening we started with OpenAI's Whisper on the processor, then moved to NVIDIA's Parakeet, which runs on the Mac's graphics cores through MLX; Whisper stays as the fallback on Windows. The voice is Kokoro, chosen over Piper because it does not sound like a machine. The name stops the radio moving the recipe on; this button does the same with one tap.

7. Why we did not train it

Say

You might expect us to have trained the model on recipes. We did not, on purpose. Gemma already cooks well in general: techniques, timings, what goes with what. What it lacks is your kitchen tonight and the exact recipe to follow, and both change daily, so they belong in a lookup, not in training.

We did build and cost the training pipeline: with LoRA, about nineteen million extra numbers beside the model, an hour or two on a gaming graphics card. It is there if the writing itself ever needs teaching.

8. Where it stands

Do

Say "Bob, stop cooking", back to Today, then stand still for the last two sentences.

Say

It still gets things wrong: the search matches words rather than meaning, and after several questions the chef can drift to the wrong step, which we measured and have a fix for.

Six days, five of us, one laptop, and we can tell you exactly which parts are weak because we measured them. Thank you, we will take questions.

Part 3. Every technical word, in one line

Learn the middle column. Say the right-hand one if someone asks on the day. Nothing here needs a diagram.

Word	What it actually is	How to say it on stage
Token	The unit a model reads and writes: roughly three quarters of a word	"It reads and writes in chunks a bit smaller than words. We call them tokens."
Context window	How much text the model can hold in mind at once	"Its memory for the current conversation. Ours holds about twelve pages; the model could hold two hundred, but every page costs waiting time, so we keep it small."
Quantisation, 4-bit	Storing each of the model's numbers in four bits instead of sixteen	"Compression. Sixteen possible values per number instead of sixty-five thousand. A ten-gigabyte file becomes four, with a small loss of precision."
GPU, Apple Silicon	The graphics processor built into the M4 chip, 10 cores, sharing the same memory as everything else	"Yes, the chip has a graphics processor built in. It shares one pool of memory with the rest of the machine, which is why a laptop can run this at all."
MLX	Apple's software for running models on its own chips	"Apple's own toolkit for running AI on Apple chips. It is the fastest way to do it on a Mac."
GGUF, llama.cpp	The file format and engine used by most other local tools, including Ollama	"The other common format, used by the tools that have to run on every machine. We use it on the Windows laptops."
LM Studio, Ollama	Two programs that load a model and answer requests on your machine	"Programs that hold the model in memory and answer our app's requests. We use one on the Mac and the other on Windows."
KV cache	The notes the model keeps about the conversation so far, so it does not reread it for every word	"Its working notes. Without them it would reread the whole conversation for every word it writes."
One shared key-value head	Eight attention heads sharing a single set of those notes instead of eight	"Normally the model keeps eight sets of notes side by side. This one keeps a single shared set, which is eight times smaller."
Sliding-window layers	Layers that look only at the last 512 tokens rather than the whole conversation	"Most of the model only looks at the last page or so, not the whole conversation. Only one layer in five looks at everything."
Why 8,192 tokens costs about 70 MB	1 KB per token per layer, over 7 full-attention layers and 28 window layers	"Those notes are about a kilobyte per word per layer. Because most layers only look at the last page, twelve pages of conversation cost about seventy megabytes rather than three hundred."
Indexing	Reading the whole corpus once, in advance, and writing a lookup table so searching is instant	"Like building the index at the back of a book, once, so you never have to read the book again to find something."
Embedding	Turning a text into a list of numbers that places it by meaning. Our prototype used the BAAI model bge-small-en-v1.5 with the ChromaDB database	"Turning each recipe into a point on a map of meaning, so you can find nearby ones. Every recipe has to pass through a small model called bge-small to get its point, and there are two million of them, which is hours of computing on a laptop. So we did not."

Word	What it actually is	How to say it on stage
BM25	The standard scoring rule for word search: rare words count more, and a match in a short text counts more	"The rule that decides which result is best in ordinary word search. Rare words count more than common ones, which is why we throw away words like salt."
The databases	SQLite with its FTS5 full-text extension for the recipe index and the Open Food Facts product list; IndexedDB in the browser for the household	"Everything is SQLite, the small database that ships with everything, using its full-text search. The household's own data lives in the browser's own storage, IndexedDB, not on a server."
RAG	Retrieval-augmented generation: search first, then let the model write using what was found	"Look it up, then write. The opposite of trying to teach the model everything in advance."
LoRA, rank 16	Freezing the model and training two small matrices per layer; the rank is how small	"A cheap way to teach a model a new habit: you freeze it and train a small set of extra numbers beside it. Rank sixteen is the size of that set: nineteen million numbers for our model, against five billion in the model itself."
Fine-tuning	Continuing to train the model on your own examples	"Retraining the model on your own data. The full version needs about eighty gigabytes of graphics memory for this model, so it is not happening on a laptop."
Schema-constrained output	The runtime refuses any token that would break the required JSON shape	"We hand it a form to fill in, and the software physically cannot let it write outside the boxes."
Hallucination	A confident statement that is not true	"When it says something wrong with complete confidence. That is what all our checks are for."
Whisper, Parakeet, Kokoro	Speech-to-text models, and a text-to-speech model	"Two small models for hearing and one for the voice, all on the laptop."

Part 4. When something breaks

A live demo will surprise us at least once. The jury cares more about how we handle it than about a perfect run. Stay calm, say what happened in one sentence, use the fallback, keep going.

What happens	What we say	What we do
The chef takes more than a minute	"It's a laptop, the model is reading its prompt. That's exactly why we cap the context at 8,000 tokens."	Wait up to 90 s. If nothing comes, go to the Recipes tab and open a recipe we saved during rehearsal.
"Bob" isn't heard in cooking mode	"That one is transcribed on the machine, so let me just press to talk."	Tap the Talk button and say the command without the name.
The camera shows nothing	"Let's use the photo we took this morning."	Import a receipt, then Pictures, then Receipts.
The receipt reads badly	"Laptop cameras aren't scanners, that's on our limits list."	Use a sample receipt.
A strange recipe	"This is why every recipe goes through checks. Let's see what they caught."	Point at the "check this before you start" box if it appears. Otherwise open a saved recipe.
No sound from the chef	"The voice is on, the room isn't."	Check the output device; the text is on screen anyway.

What happens	What we say	What we do
The backend crashes	"The core of the app doesn't need the server, look."	Show the Kitchen and Shopping screens, which still work. Restart with <code>./run.sh --no-reload</code> .

Fallbacks.

Part 5. Questions to expect

Question	Short answer
Is 128,000 tokens a lot?	It is about 98,000 words, roughly a 200-page paperback. We deliberately use 8,192, about twelve pages, because the model reads the whole request before writing anything and every page costs waiting time.
Why load only 8,192 then?	Speed, not memory. Twelve pages of context is more than a recipe request needs, and reading more would make every answer slower on a laptop.
What is the cache, and why does 8,192 tokens cost 70 MB?	The notes the model keeps about the conversation so it does not reread it for every word. They cost about a kilobyte per token per layer. This model keeps a single shared set of notes rather than eight, and most of its layers only look back one page, so twelve pages cost about 70 MB instead of roughly 300.
Does an M4 have a GPU?	Yes, built into the chip: ten graphics cores on this machine. It shares one pool of memory with the processor, which is why a laptop with no separate graphics card can run the model.
What is MLX, and what is GGUF?	MLX is Apple's toolkit for running models on Apple chips, and the format our model comes in. GGUF is the other common format, used by the portable engine behind most local tools, and what the Windows machines use.
What does 4-bit quantised mean?	Each number is stored in four bits instead of sixteen: sixteen possible values instead of sixty-five thousand. The weight file goes from 10.25 GB to 4.34 GB. What it needs in memory is less again, because roughly half of this model is lookup tables read a row at a time rather than held resident.
What does indexing mean?	Reading all 2,119,522 recipes of the Kaiser1308/CookingRecipes dataset once, in advance, and writing a lookup table, like the index at the back of a book. After that, searching does not touch the recipes themselves.
What is an embedding, and why would it take hours?	Turning each recipe into a list of numbers that places it by meaning, so similar recipes sit near each other. Every recipe has to pass through a small model, bge-small-en-v1.5, and two million of them is hours of computing on a laptop, which is why we kept word search instead. The prototype that does it with ChromaDB is still in the repository.
What is BM25?	The standard scoring rule for word search: rare words count for more than common ones, and a match in a short text counts for more than in a long one.
Which database?	SQLite with its FTS5 full-text search extension: one file holds the index of Kaiser1308/CookingRecipes, another the products from Open Food Facts. The household's own data lives in the browser's IndexedDB, never on a server.
What is LoRA, and what does rank 16 mean?	A cheap way to teach a model a new habit: you freeze the model and train a small set of extra numbers beside it. The rank is the size of that set. At rank 16 on our model it is 18.9 million numbers against 5.1 billion in the model, about a third of a percent.
Why do you quote a 7-billion model when you run Gemma E2B?	Because our training scripts were written for Qwen 7B before we moved the demo to Gemma. On Gemma the same method is 18.9 million trainable numbers and a full retrain would need about 80 GB of graphics memory; on the 7B it is 40 million and about 120 GB. Both are in the report.

Question	Short answer
Why didn't you train it on recipes?	Because Gemma already cooks well in general; what it lacks is your kitchen tonight and the exact recipe, and both change daily. Training freezes knowledge at training time, a lookup is always current, and a looked-up recipe comes with its source. We built the pipeline anyway and costed it: 18.9 million extra numbers with LoRA on Gemma, an hour or two on one gaming GPU.
What did you use before, and why did you change?	Chat model: a cloud model on day one, then Qwen 2.5 7B (too much memory beside the voice), then Qwen 3.5 9B (reasoning mode, 26 minutes on one recipe), then Gemma 4 E2B. Listening: OpenAI's Whisper on the processor, replaced by NVIDIA's Parakeet on the graphics cores through MLX; Whisper stays as the fallback on Windows. Voice: Kokoro from the start, chosen over Piper for how it sounds.
Why not just use ChatGPT?	Running it ourselves was the point of the exercise, and it changes the product: your kitchen never leaves the laptop, there is no cost per question, and it works without internet.
How do you stop it inventing things?	It writes into a fixed form the software will not let it break, we hand it real recipes to work from, and then ordinary code checks ingredients, quantities, times and whether the rice is ever cooked. What is still wrong is shown on the recipe, not hidden.
How did you evaluate it?	Not with the usual training measure, because there is no single right answer to "dinner from this fridge". We measured how often recipes pass our checks, how long retrieval and generation take, and ran small controlled tests, such as the one that showed the chef drifting to the wrong step.

Part 6. Numbers to know by heart

Figure	Value
Machine	MacBook Air, Apple M4: 10 processor cores, 10 graphics cores, 16 GB shared by everything
Model	google/gemma-4-E2B-it, run as the 4-bit MLX build from lmstudio-community: 5.1 billion numbers in total, about 2.3 billion effective, 35 layers
Size	a 10.25 GB weight file as published, 4.34 GB for the 4-bit build we run; roughly 2 to 4 GB resident, depending on the context window
Speed	about 15 words a second; a full recipe is roughly 700 tokens, so 45 to 50 seconds
Context	the model can hold about 200 paperback pages; we give it 12, and keep a third of that for the answer
Its working notes	about 70 MB for those 12 pages, because it keeps one shared set of notes and most layers look back only one page
Recipe corpus	Kaiser1308/CookingRecipes: 2,119,522 indexed, searched with SQLite FTS5; 3.3 seconds a query down to a few milliseconds
Receipt products	Open Food Facts, filtered to France: 483,210 products, 329,783 matched to one of our 3,052 catalogue ingredients
Teaching it a habit (LoRA)	18.9 million extra numbers on Gemma 4 E2B, 0.37 %; retraining all of it would need about 80 GB of graphics memory
The same on the model our scripts target	Qwen/Qwen2.5-7B-Instruct: 40.4 million, 0.53 %, and about 120 GB for a full retrain
Voice	Parakeet TDT 0.6B hears (MLX build), Kokoro-82M speaks, both on the laptop; a short phrase synthesises in 0.15 to 0.25 s

Figure	Value
The team	five people, six days